UNIT 2, BUILDING INTELLIGENT AGENT WORKFLOWS

LECTURE 3

# API Integration

The tools leave your laptop. They reach real services over the network now, where every failure from last lecture stops being an example and becomes an ordinary Tuesday afternoon.

CSE476 | Rohit Bharti

SECTION 00

# From your machine to the network

Last lecture the tools were local. Now they call the outside world.

RECALL

# What changed since last lecture

LECTURE 2

## Tools on your machine

A function you wrote, running in your process. When it failed, it failed in ways you controlled, and you handled them with call_tool.

LECTURE 3

## Tools reaching real services

The function now makes a network request to somebody else's server. It can be slow, refuse you, change its mind about its own format, or simply not be there. You control none of it.

THE GOOD NEWS

You already built the defence. Last lecture's call_tool handles the failures. Today is about one layer below it: the wrapper that turns a real, messy API into the clean tool call_tool then protects. Two thin layers, each with one job.

# Today

- What a real API actually hands you, and why you never show it to the model.
- The wrapper: a thin translating layer, and why it is the most important idea in the lecture.
- The three things every real request needs that beginners forget.
- Why 401, 429, and 503 are three different problems, not one.
- The seam that lets you test all of this without ever touching the network.

HOLD ON TO THIS

Same weather assistant as last lecture, because now you get to see what the weather tool looked like on the inside all along: a messy third party API, wrapped in a layer that hid the mess from you.

SECTION 01

# What a real API hands you

It is never the clean thing you wanted.

# The raw payload, in all its glory

```
{
  "loc": { "n": "Delhi", "cc": "IN" },
  "cur": {
    "tmp_k": 311.15,        // Kelvin. KELVIN.
    "hum_pct": 30,
    "cond_cd": 721          // a code. 721 means "haze".
  }
}
```

IMAGINE HANDING THIS TO THE MODEL

Nested objects, abbreviated keys, temperature in Kelvin, weather as a numeric code. If you pass this straight to the model, you are asking it to know that 311.15 Kelvin is hot and that 721 means haze. It might. It might not. And you are paying for every token of that noise on every single call.

# The rule this lecture is built on

> **Never let the shape of an external API leak into your agent.**

Between the messy API and your clean agent, you put a thin layer that translates one into the other. The API can be as ugly as it likes. The model only ever sees the tidy sentence that comes out the other side.

THIS PATTERN HAS A NAME

Engineers call it an anti-corruption layer. The idea is that a foreign system's messiness should not be allowed to corrupt the clean design of yours. You are about to build a tiny one, and the discipline it teaches will outlast every specific API you ever touch.

SECTION 02

# The wrapper

A thin function that turns mess into a clean tool.

# The wrapper, in full

```
def get_weather(city):
    try:
        raw = client.get_json("weather", {"q": city})
    except ApiError as e:
        return f"Could not get weather: {e}"   # readable

    celsius   = round(raw["cur"]["tmp_k"] - 273.15)
    condition = COND[raw["cur"]["cond_cd"]]     # 721 -> "haze"
    return f"{loc}: {celsius}C, {condition}."   # clean
```

READ THE FIRST AND LAST LINES

City in. One readable sentence out. Everything between, the Kelvin, the codes, the nested keys, the network errors, is translated here and never escapes. The model sees a tool as simple as any you wrote in Unit 1, and has no idea a messy API is hiding behind it. That is the whole job.

# The schema the model sees

```
"name": "get_weather",
"description": "Get current weather for one city as a
   short readable summary. Known: Mumbai, Delhi, Jammu.",
"parameters": { "city": "City name, e.g. Delhi" }
```

HOLD ON TO THIS

Notice what is absent. No mention of Kelvin, no condition codes, no nested payload, no API at all. The schema describes the clean tool, not the messy service behind it. The model programs against the tidy version, exactly as it should.

THE PAYOFF

If you swap the weather provider next year for a different messy API, you rewrite the wrapper and change nothing else. The schema stays the same, the model behaves the same, every notebook still runs. The wrapper is the shock absorber for a world of APIs that change without asking you.

SECTION 03

# The three things beginners forget

Timeout, credentials, and status codes.

# One: always set a timeout

WITHOUT A TIMEOUT

## Wait forever

The default for many HTTP libraries is to wait indefinitely. One slow server, and your agent hangs, your user waits, and nothing ever errors. The worst failure, because it is silent and permanent.

WITH A TIMEOUT

## Fail in five seconds

A bounded wait, then a clean ApiError your wrapper turns into a sentence. The user gets "I could not reach the service" instead of a spinner that never stops.

THE SAME LESSON AS max_steps

In Lecture 1 an uncapped loop could run forever. Here an uncapped request can wait forever. Same failure, different layer. Every place your code waits on something it does not control needs a bound. Learn to feel the absence of one.

# Two: the key goes in a header, never the URL

THE MISTAKE

## Key in the URL

api.com/weather?key=secret123. It works, so beginners do it. But URLs end up in server logs, browser history, error messages, and analytics. Your secret is now in five places you forgot about.

THE HABIT

## Key in a header

Authorization: Bearer secret123. Headers are not logged by default and do not travel in the visible address. Same result, none of the leaks. Cost of doing it right: zero.

HOLD ON TO THIS

This is a Unit 6 security lesson arriving three units early, because the habit is free to form now and expensive to retrofit later. Your test even checks that the key never appears in the requested URL.

# Three: 401, 429 and 503 are three problems

| | | |
|---|---|---|
| 401 | **Unauthorized** | Your problem. The key is wrong or missing. Retrying will never help. Fix the credential. |
| 429 | **Too Many Requests** | A timing problem. You went too fast. Back off and retry, exactly the backoff from last lecture. |
| 503 | **Server Error** | Their problem. The service is down or struggling. Retry later, and if it persists, fail honestly. |

### WHY NOT JUST CATCH THEM ALL THE SAME WAY

Because each one calls for a different response, and collapsing them into one "error" throws away the information you need to act. Retrying a 401 is pointless. Not retrying a 503 is giving up too early. The status code is telling you whose fault it is, and whose fault it is decides what you do.

SECTION 04

# The seam

How to test network code without a network.

# The problem with testing network code

THE OBVIOUS WAY

## Call the real API in your test

Slow, because it hits the network every run. Flaky, because the service might be down. Impossible to test a 503, because you cannot make a real server fail on command. And it costs money or quota every time.

THE BETTER WAY

## Put a seam where the network is

Your code depends on a transport it is handed, not on a hard-coded network library. In production you hand it a real one. In tests you hand it a fake that fails exactly how you tell it to.

THIS IS WHY THE HTTP CLIENT TAKES A transport

It looks like a small design choice. It is the thing that lets every network fault in this whole lecture, timeouts, 401s, 429s, broken JSON, be reproduced instantly, offline, in a test that never flakes. You will meet this idea properly in Unit 5. It is called dependency injection, and it is worth its weight.

# The fake, in practice

```
# production
client = HttpClient(transport=RealHttp(), base_url=...)

# a test that proves we survive a 429
fake = FakeTransport(status=429)
client = HttpClient(transport=fake, base_url=...)
assert "back off" in get_weather("Delhi").lower()

# a test that proves a timeout does not crash us
fake = FakeTransport(raise_timeout=True)
```

EVERYTHING ABOVE THE SEAM IS IDENTICAL

The only line that changed is which transport you handed in. The wrapper, the tool, the loop, all untouched. That is the mark of a good seam: the fake and the real thing are interchangeable, so a test proves something true about production, not about the test.

# The whole picture

| | |
|---|---|
| THE MODEL | Sees one clean tool: city in, sentence out |
| THE WRAPPER | Translates mess to clean, both directions |
| THE HTTP CLIENT | Timeout, auth header, status handling |
| THE TRANSPORT SEAM | Real network in production, fake in tests |
| THE MESSY API | Kelvin, codes, nested keys, and moods |

Clean at the top, messy at the bottom, and a thin translating layer in between keeping them apart. Each layer has exactly one job.

# Hold on to this

**The messy API stops at the wrapper. The model only ever sees clean.**

| | |
|---|---|
| **Never leak the API shape** | Wrap it. Translate Kelvin and codes into a plain sentence. |
| **Always set a timeout** | An unbounded wait is the silent cousin of the runaway loop. |
| **Key in a header, not the URL** | URLs get logged. Headers do not. The habit is free. |
| **401, 429, 503 are three problems** | Your fault, your timing, their fault. Owner decides action. |

# Before next session

| | | |
|---|---|---|
| DO | **Wrap a second endpoint** | The weather API surely has a forecast endpoint too. Add a get_forecast tool that wraps it, reusing the same HttpClient. Notice how little new code it takes once the client exists. |
| DO | **Test a failure without a network** | Write a test using FakeTransport that proves your agent says something sensible when the API returns 503. You will not touch the real network once. |
| THINK | **Spot the leak** | Find an API you have used whose raw response you passed straight into your code. What would break the day that API changed one field name? The wrapper is what would have saved you. |

ON THE SECOND ONE

Being able to test the failure path without triggering a real failure is a genuine professional skill, and it is rare. Most people only test the happy path, because the failure path is hard to reach. The seam makes it easy. Use it.

# Next session

Unit 2, Lecture 4

## Workflow Chaining and Event-Driven Systems

You can now call one service safely. Next time we chain several into a pipeline, where the output of one step feeds the next, and we look at systems that react to events rather than waiting to be asked. This is where single tool calls grow into real automation, and where the workflow versus agent decision from Lecture 1 comes back with more at stake.

COME WITH

Your forecast tool wrapped, your 503 test written, and one API leak you found in your own past code.